

## **Exercise 3: Your React Native Messages Directory**

**Student Name:** Yingtian Wu

**Student ID:** 1297928

### **1. Project Description**

This project is a React Native message directory application developed for Exercise 3. The application displays a list of important message directories. Each directory has a title, local image icon, color theme, short description, message count, and latest message preview.

When the user selects a directory, the app opens a detail screen and displays the stored messages for that directory. The user can also return to the main directory screen by pressing the back button.

The app includes the following directories:

- You
- Home
- Love
- Family
- Friends
- School

The goal of this project is to demonstrate React Native app development using TypeScript, reusable components, local image assets, responsive layout, and interaction through clickable directory cards.

### **2. Technologies Used**

The following technologies and tools were used:

- React Native
- Expo
- TypeScript
- IntelliJ IDEA
- Android Emulator
- Chrome Web
- React Native Core Components
- Local PNG image assets

### **3. Main Features**

The application includes the following features:

1. Displays a message directory home screen.
2. Shows six important directories: You, Home, Love, Family, Friends, and School.
3. Uses local image assets for each directory.
4. Shows message count for every directory.
5. Shows the latest message preview on each directory card.
6. Opens a stored messages detail screen when a directory is clicked.
7. Displays stored messages with sender name, message content, and time.
8. Includes a back button to return to the main directory screen.
9. Uses TypeScript types for structured data.
10. Uses reusable React Native components.
11. Supports responsive layout for different screen sizes.
12. Runs on both Chrome Web and Android Emulator.

## 4. Project Structure

The project structure is organized as follows:

exercise3\_yingtianwu/

```
|— App.tsx
|— index.ts
|— app.json
|— package.json
|— package-lock.json
|— tsconfig.json
|— assets/
|   |— icons/
|   |   |— you.png
|   |   |— home.png
|   |   |— love.png
|   |   |— family.png
|   |   |— friends.png
|   |   |— school.png
|   |— screenshots/
|   |   |— web-home.png
|   |   |— android-home.png
|   |   |— android-detail-school.png
|— src/
|   |— types.ts
|   |— data/
|   |   |— directories.ts
|   |— components/
|   |   |— DirectoryCard.tsx
|   |   |— MessageCard.tsx
```

### File Descriptions

#### App.tsx

This is the main application file. It manages the selected directory state, displays the home screen, displays the detail screen, and controls the navigation between the two views.

#### index.ts

This file registers the main app entry point for Expo.

**app.json**

This file contains Expo project configuration.

**package.json**

This file contains project dependencies and scripts for running the app.

**tsconfig.json**

This file contains TypeScript configuration.

**src/types.ts**

This file defines the TypeScript types used in the app, including Directory and Message.

**src/data/directories.ts**

This file stores all directory data, including titles, colors, local images, descriptions, and stored messages.

**src/components/DirectoryCard.tsx**

This reusable component displays each directory card on the home screen.

**src/components/MessageCard.tsx**

This reusable component displays each stored message on the detail screen.

**assets/icons/**

This folder stores the local PNG images used for the six directory icons.

**assets/screenshots/**

This folder stores screenshots used in this README file.

## **5. Installation and Configuration**

Before running the project, Node.js and npm must be installed on the computer.

To install the project dependencies, open the project folder in IntelliJ IDEA and run the following command in the terminal:

*npm install*

This command installs all required packages listed in package.json.

## **6. How to Run the Application**

### **6.1 Run on Chrome Web**

To run the application in Chrome Web, use the following command:

*npm run web*

After running the command, Expo will show a local web URL in the terminal. Open the URL in Chrome. The web version of the app will display the message directory home screen.

The port number may be different depending on the current computer environment.

## 6.2 Run on Android Emulator

To run the application on Android Emulator:

1. Open IntelliJ IDEA.
2. Open the Device Manager.
3. Start the Pixel 6 API 35 Android Emulator.
4. After the emulator is fully started, run the following command:

*npm run android*

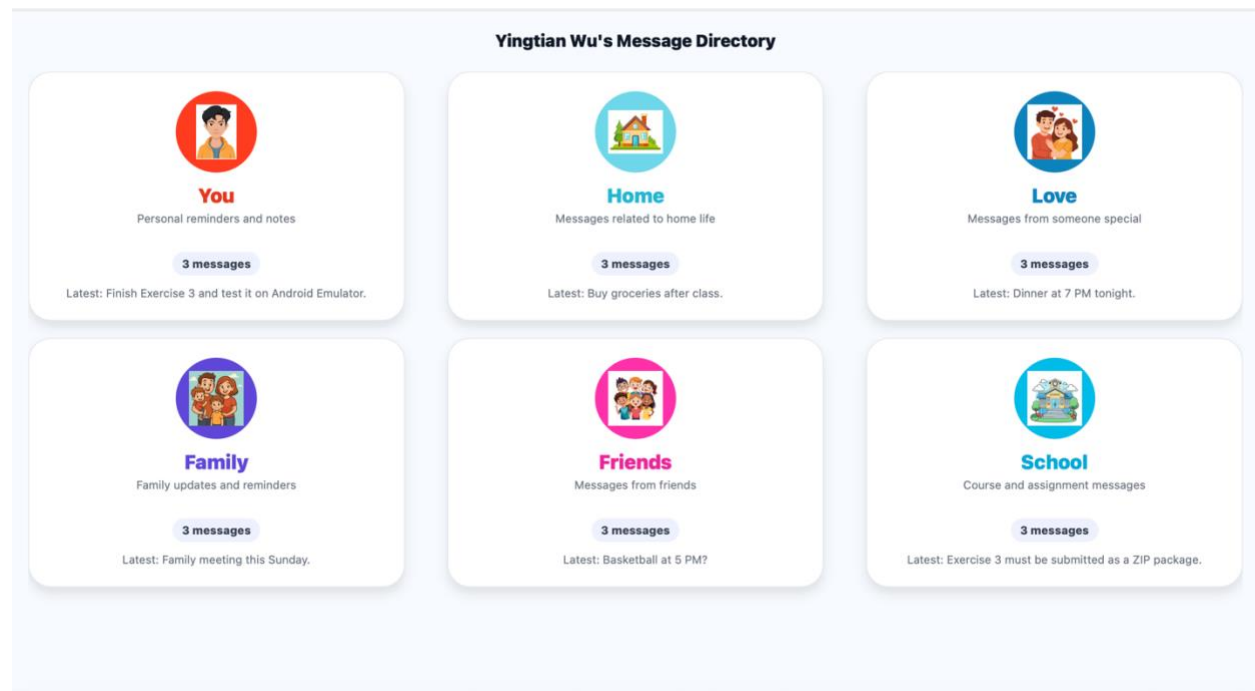
Expo will build and open the app on the Android Emulator.

The app was tested using:

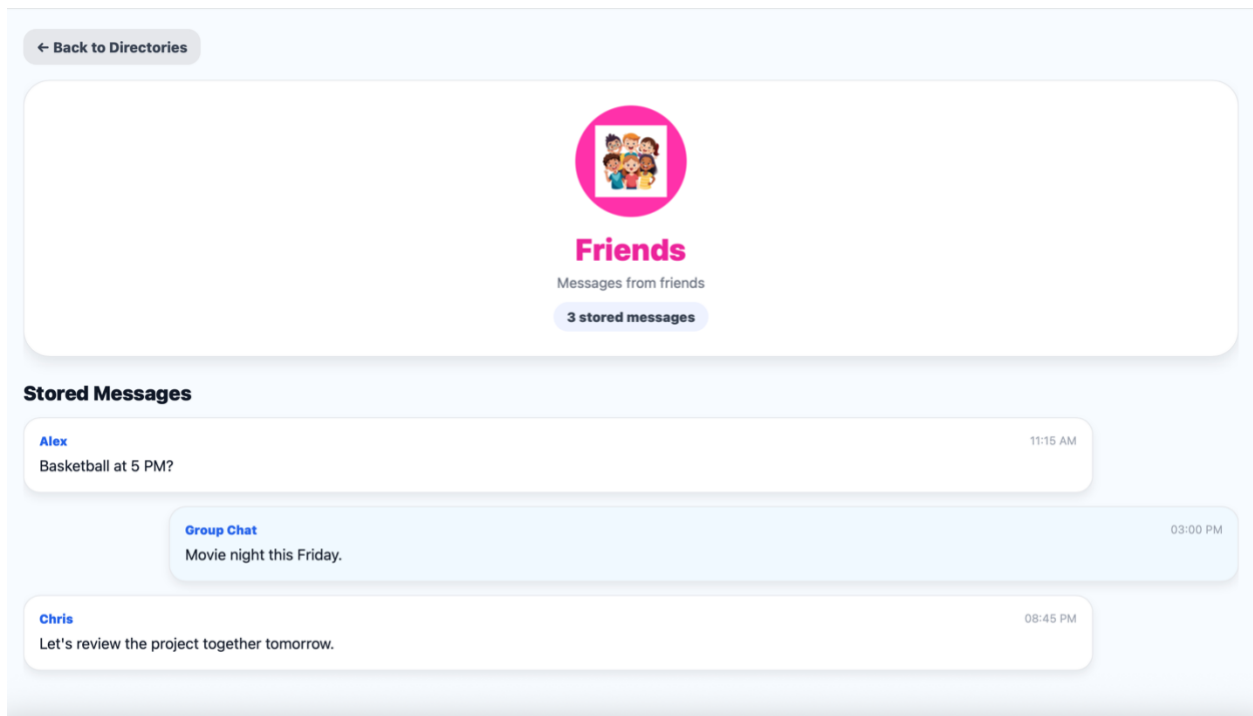
Pixel 6 API 35 Android Emulator

## 7. Screenshots

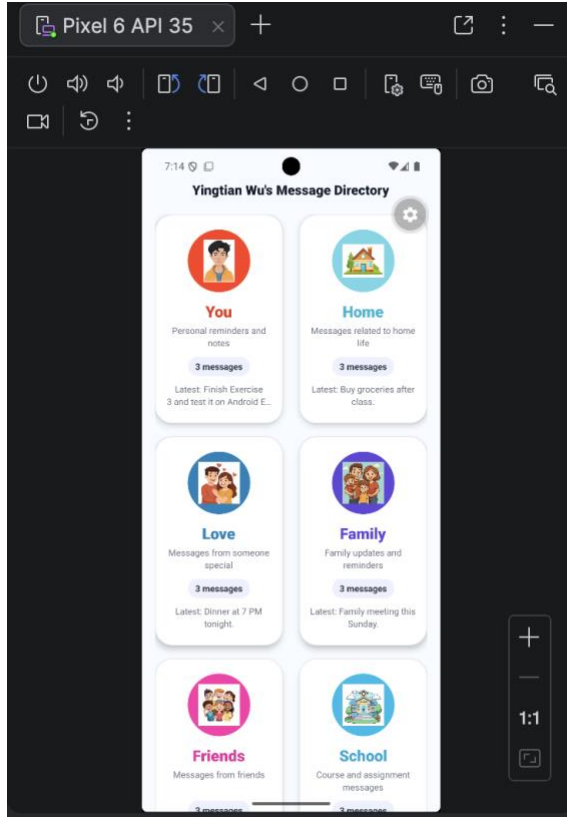
### Screenshot 1: Chrome Web Home Screen



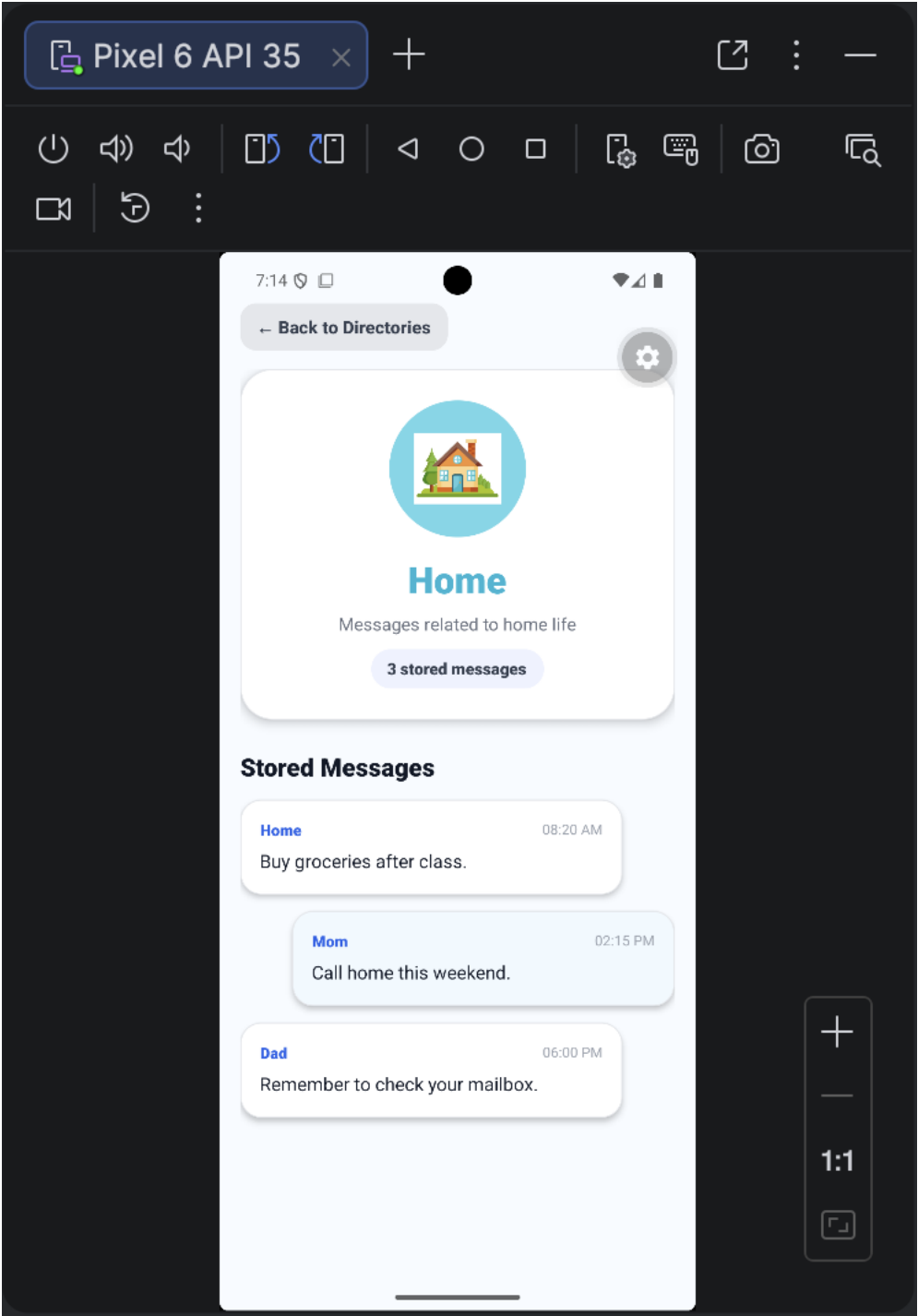
## Screenshot 2: Chrome Web Directory Detail Screen



## Screenshot 3: Android Emulator Home Screen



Screenshot 4: Android Emulator Directory Detail Screen



## **8. Design Details**

The app uses a clean card-based layout. Each directory is displayed as a rounded card with a local image, title, short description, message count, and latest message preview.

The detail screen uses the selected directory color and image to keep the design consistent.

The layout uses FlatList with dynamic columns. On wider screens, the app displays three columns. On smaller mobile screens, the app displays two columns.

## **9. Conclusion**

This React Native application satisfies the requirements of Exercise 3 by displaying a list of important message directories and showing stored messages when a directory is selected.

The project uses TypeScript, reusable components, local images, responsive layout, Chrome Web testing, and Android Emulator testing. The application is organized with a clear project structure and can be configured and run using the commands provided in this README.